

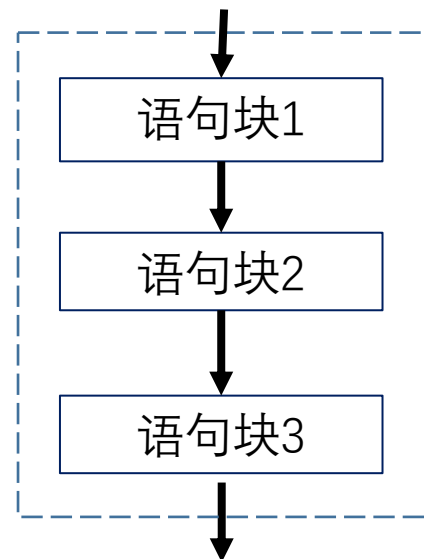
第3章 选择与循环

Python程序结构

- 顺序结构：
 - 按照先后顺序，逐个执行
- 选择结构
- 循环结构
- 异常处理结构(第8章)

```
try:  
    # 执行代码块  
except TypeError as e:  
    # 异常处理  
except (IndexError, ValueError):  
    # 异常处理
```

复旦大学版权所有

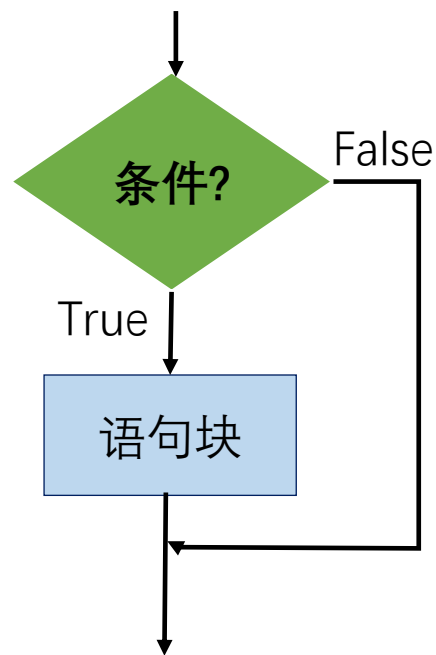


选择结构

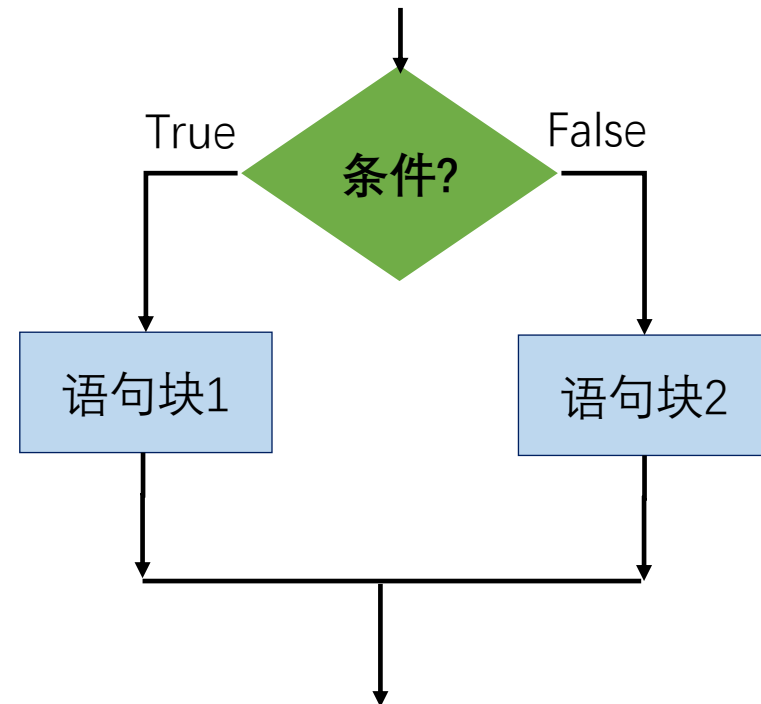
如果.....,
就.....,
否则.....。

3.2 选择结构

1. 单分支

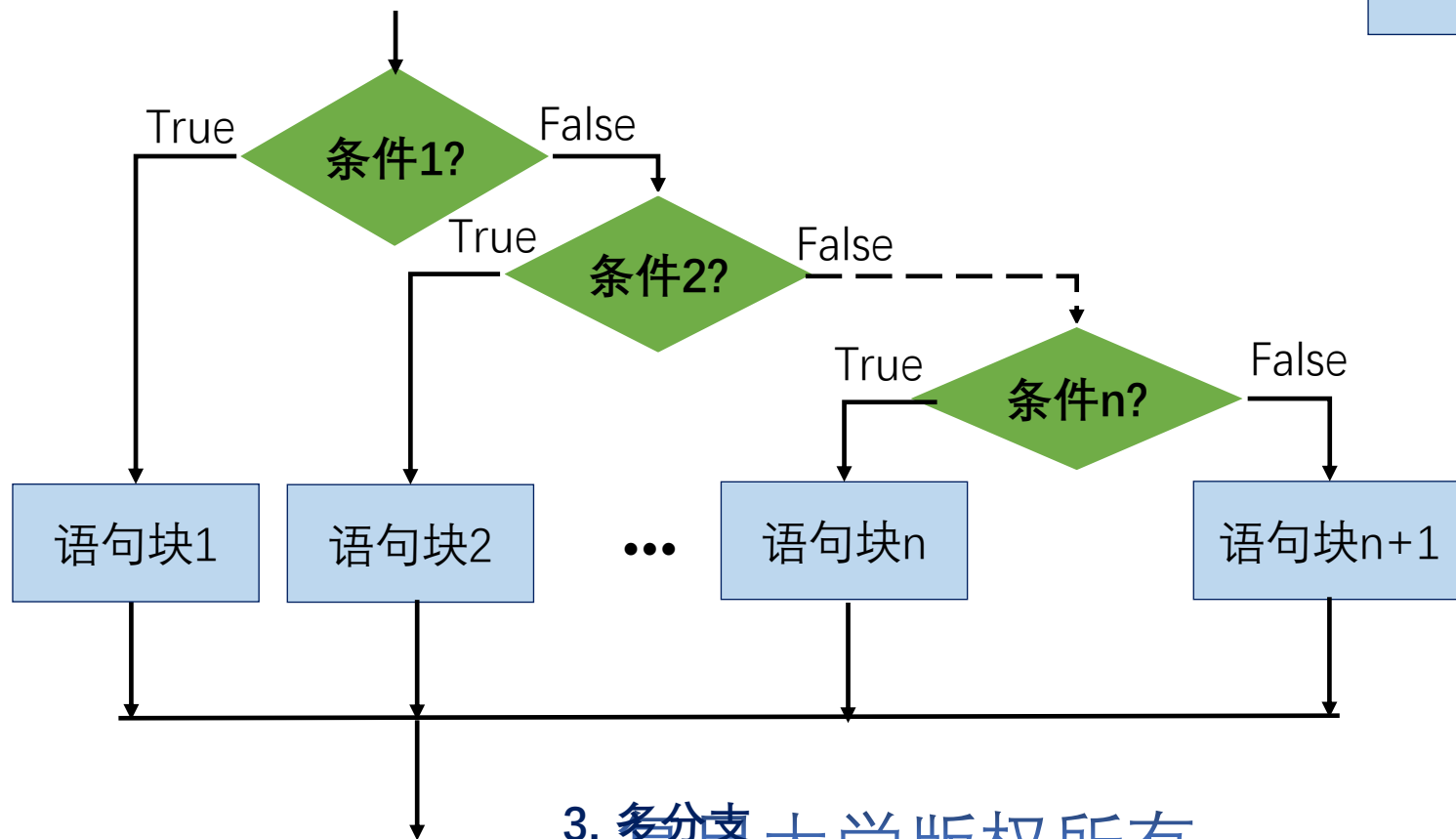


2. 双分支



3. 多分支

复旦大学版权所有



3.2.1 单分支选择结构

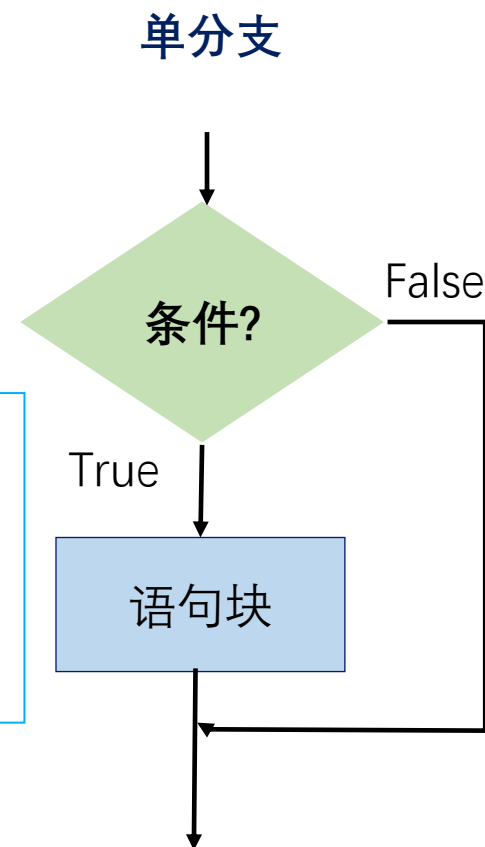
if 条件表达式: # 其他语言要求条件表达式有括号
语句块

例1: 提示用户输入两个数，然后按照从小到大顺序输出两个数

```
a, b = eval(input( '请输入两个数 ( 以英文逗号分隔 ) :'))  
if a>b:  
    a,b=b,a  
print(a,b)
```

例2: main函数的调用

```
if __name__ == '__main__':  
    main()
```



布尔值 (Bool)

True

1

其他

False

0

**None、空字符串、空列表、
空元组、空字典、空集合**

条件表达式

- $1==1$
- $1==2$
- $1=='1'$

- $1!=1$
- $1!=2$
- $1!='1'$

3.1 条件表达式

- 条件表达式: 用在if/while等作为决定分支或用于循环的条件时进行真值判断
- 关系运算符：>、<、==、<=、>=、!= 其值为False 或True(比较等运算结果)
- 测试运算符：成员关系测试的in、not in；同一对象测试的is、is not
- bool(expr → obj)
 - 如果其值为0 0.0（数值类型）、None、空序列对象，则取值为False
 - 所谓空序列对象obj指的是其长度为0，即len(obj) == 0
 - "" [] () set() {} range(0)等取值False
 - 否则取值为True，即非0、非空对象等为True。
1 -5 "abc" [1,2] (1,) {one:'one'}
- 布尔（bool）类型取值为True、False，分别对应整数1和0

```
>>> int(True), int(False)
```

```
(1, 0)
```

3.1 条件表达式:主要运算符

- 关系运算符： $>$ 、 $<$ 、 $==$ 、 $<=$ 、 $>=$ 、 $!=$ ，可以连续使用，连续使用时表示这些条件都满足(AND)

```
>>> print(1<2<3)
```

```
True
```

```
>>> print(1<2>3)
```

```
False
```

```
>>> print(1<3>2)
```

```
True
```

- 逻辑运算符：`and`、`or`、`not`，注意短路求值

3.1 条件表达式：逻辑(布尔)运算符

运算符	说明	实例	结果
逻辑非 <u>not expr</u>	expr为真值时返回False, expr为假值时返回True.	not 4, not False, not True not [1,2], not "	(False, True, False) (False, True)
逻辑与 <u>expr1 and expr2</u> <u>and expr3</u>	按顺序计算表达式的值, 前面表达式为假时返回该表达式的值, 后面表达式不再计算。否则计算下一个表达式, 如果到达最后一个表达式(即前面都为True), 则返回该表达式	[] and True 4 and {} and 3.14 [1] and (2,) and "3"	[] { "3"
逻辑或 <u>expr1 or expr2</u> <u>or expr3</u>	按顺序计算表达式的值, 前面表达式为真时返回该表达式的值, 后面表达式不再计算。否则计算下一个表达式, 如果到达最后一个表达式(即前面都为False), 则返回该表达式	[] or True 4 or {} or 3.14 [1] or (2,) or "3" [] or [1,2] or None False or [] or [1,2,3]	True 4 [1] [1, 2] [1, 2, 3]

短路逻辑(short-circuit logic) **and : 返回第一个假(None、空或者数值0)的表达式或者最后一个表达式**
惰性求值(lazy evaluation) **or: 返回第一个真(非None、非空或者非0)的表达式或者最后一个表达式**

3.1 条件表达式：逻辑(布尔)运算符

and : 返回第一个假(**None**、空或者数值0)的表达式或者最后一个表达式

• **and** : 逻辑与运算 expr1 and expr2 and expr3

- 只有所有条件(表达式)为真才为真值
- and运算采用左结合律，即从左到右进行运算
- 短路逻辑(short-circuit logic) 或者惰性求值(lazy evaluation)
 - 如果前面有假，不管后面的条件(表达式)怎样，最终也是假值
 - 后面的表达式不需要进行计算
- 注意与其他语言不同，**and/or运算返回的结果并不一定是True或者False，而是返回某个表达式的值**(用于if/while中的条件时再进行真值判断)
 - AND 返回的是最后一个决定真值判断的**表达式的值**，即**返回第一个假(None、空或者数值0)的表达式或者最后一个表达式**

```
False and True
4 and False and 3.14
[1] and (2,) and "3"
```

```
False
{}
"3"
```

3.1 条件表达式：逻辑(布尔)运算符

and：返回第一个假(**None、空或者数值0**)的表达式或者最后一个表达式

问题：分数及格时输出信息：其通过了测试

```
scores = {'bob':65, 'jim': 80}
key = 'bob'

score = key in scores and scores[key] # 65
# 如果key=='nobody', score为False (key in scores)

if score >= 60:
    print('%s passed the exam' % key )
```

```
scores = {'bob':65, 'jim': 80}
key = 'bob'

if key in scores and scores[key] >= 60:
    print('%s passed the exam' % key )
```

```
if scores.get(key,0) >= 60:
    print('%s passed the exam' % key )
```

为真要求 key in scores以及scores[key]
>= 60两个条件满足

3.1 条件表达式：逻辑(布尔)运算符

- **or: 返回第一个真（非None、非空或者非0）的表达式或者最后一个表达式**
 - 只要其中任一条件为真，则为真值，否则为假值
 - 前面条件如果为真，则返回第一个真的表达式或者最后一个表达式

```
>>> param1, param2 = 'Steve', ''
>>> name1 = param1 or 'UNKNOWN'
>>> name2 = param2 or 'UNKNOWN'
>>> print(name1, name2)
Steve UNKNOWN
```

对象为空对象(字符串、列表等)时采用缺省值

```
name = input('Please enter your name: ') or '<unknown>'
```

运算符优先级

从低到高

- 优先级越低，越最后计算
- 除了**为右结合外，其他运算符都是左结合
- lambda < if-else < 逻辑运算符(or, and, not) < 比较运算符 < 两元位运算 < 算术运算 < 一元运算 < 求幂运算 < 下标、切片、函数调用、字面量定义等
- 赋值不是运算符，其采用右结合方式
- 首先定义对象，然后才可以运算 → 字面量优先级最高
- 函数调用、下标和切片、属性等返回对象 → 优先级次之
- 有了对象，可以进行计算 → 算术运算符次之，一元的优先级更高
- 计算后可以进行比较 → 比较运算符比算术运算符更低
- 比较以及成员判断的结果为True/False，可以进行逻辑运算 → 逻辑运算比比较运算符低，单目逻辑运算符(not)更高
- 三元运算符及lambda表达式优先级最低

运算符	描述
lambda	Lambda表达式
if-else	3元条件表达式
or	布尔“或”
and	布尔“与”
not x	布尔“非”，
in, not in	成员测试，与下面同一性和比较相同优先级
is, is not	同一性测试
<, <=, >, >=, !=, ==	比较
	按位或
^	按位异或
&	按位与
<<, >>	移位
+, -	加法与减法
*, /, //, %	乘法、除法与取余
+x, -x, ~x	正负号，按位求反，一元运算或单目运算
**	求幂
x.attribute	属性
x[index] x[index:index]	下标和切片
f(arguments...)	函数调用
(expression,...)	元组字面量
[expression,...]	列表字面量
{key:datum,...}	字典字面量
'expression,...'	字符串字面量

3.1 条件表达式：短路逻辑

- and和or在前面的表达式真值判断时就已确定最终结果时后面的表达式不会再计算

`if f1() or f2(): pass`

如果f1返回真值（非空或非0），Python将不再执行f2，若要保证两个函数都执行，需要在or之前调用他们

`tmp1, tmp2 = f1(), f2()`

`if tmp1 or tmp2:`
`pass`

- `any(iterable)`: iterable中任一元素的真值为真时返回True，都为假返回False。长度为0时返回False
- `all(iterable)`: iterable中所有元素都为真时返回True，否则任一元素为假返回False。长度为0时返回True

`>>> any([1,0,0])`

True

`>>> any((0,0,0))`

False

`>>> any({})`

False

`>>> all([1,0,0])`

False

`>>> all([1,2,3])`

True

`>>> all(())`

True

- any和all都不是惰性求值，相当于非惰性求值的or和and

```
if any((f1(), f2())) :  
    pass
```

```
if all((f1(), f2())) :  
    pass
```

3.1 条件表达式：不允许赋值运算符 “=”

不允许赋值运算符 “=”

- 避免可能的Bug：误将关系运算符 “==” 写成赋值运算符 “=”

```
>>> if a=3:pass
```

```
SyntaxError: invalid syntax
```

```
>>> if (a=3) and (b=4):pass
```

```
SyntaxError: invalid syntax
```

3.2.2 双分支结构

if 条件表达式:

语句块1

else:

语句块2

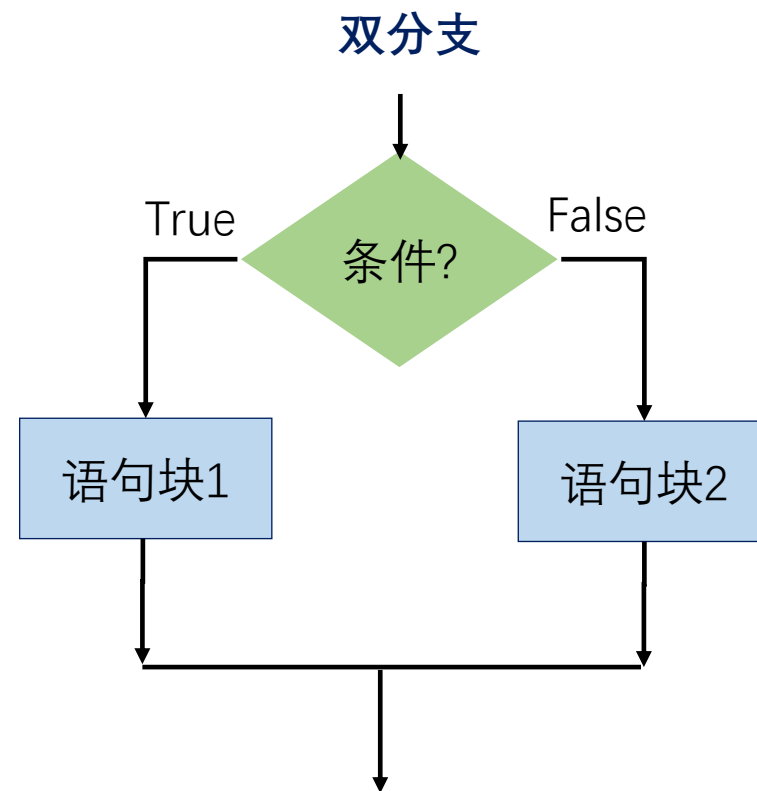
```
x = int(input("请输入整数:"))
```

```
if x % 2 == 0:
```

```
    print(x, '是一个偶数')
```

```
else:
```

```
    print(x, '是一个奇数')
```



```
chTest = ['1', '2', '3', '4', '5']
```

```
def test_nonempty(obj):
```

```
    if obj: #如果非空, 不需要 obj != []
        print(obj)
```

```
    else:
```

```
        print('Empty')
```

```
test_nonempty(chTest)
```

复旦大学版权所有

['1', '2', '3', '4', '5']

3.2.2 双分支结构

- Python还支持if|else三元表达式

value1 if condition else value2

当条件表达式condition的值为True时，表达式的值为value1，否则值为value2。

- 注意这里为短路计算，并不会同时对value1 和value2 求值。

```
>>> a = 5
>>> print(6) if a>3 else print(5)
6
>>> print(6 if a>3 else 5)
6
>>> b = 6 if a>13 else 9
>>> b
9
```

```
if condition:
    return value1
else:
    return value2
```

3.2.2 双分支结构: if/else三元运算符, 惰性求值

```
>>> a = 5
```

```
>>> x = math.sqrt(9) if a>3 else random.randint(1, 100) #此时还没有导入math模块
```

TypeError: name 'math' is not defined

```
>>> import math
```

```
>>> a = 2
```

```
>>> x = math.sqrt(9) if a>3 else random.randint(1, 100)
```

TypeError: name 'random' is not defined

```
>>> import random
```

```
>>> x = math.sqrt(9) if a>3 else random.randint(1, 100)
```

3.2.4 选择结构的嵌套

D [60-70) C [70-80) B [80-90) A [90-100) A 100

成绩从百分制变换到等级制

```
if 条件表达式1:
    语句块1
    if 条件表达式11:
        语句块2
    else:
        语句块3
else:
    if 条件表达式21:
        语句块4
    else:
        语句块5
```

```
def grade(score):
    if score > 100 or score < 0:
        return 'wrong score.must in [0,100].' # None
    else:
        if score >= 90:
            return 'A'
        else:
            if score >= 80:
                return 'B'
            else:
                if score >= 70:
                    return 'C'
                else:
                    if score >= 60:
                        return 'D'
                    else:
                        return 'F'
```

注意：缩进必须要正确并且一致。

3.2.4 选择结构的嵌套

D C B A A
[60-70) [70-80) [80-90) [90-100) 100

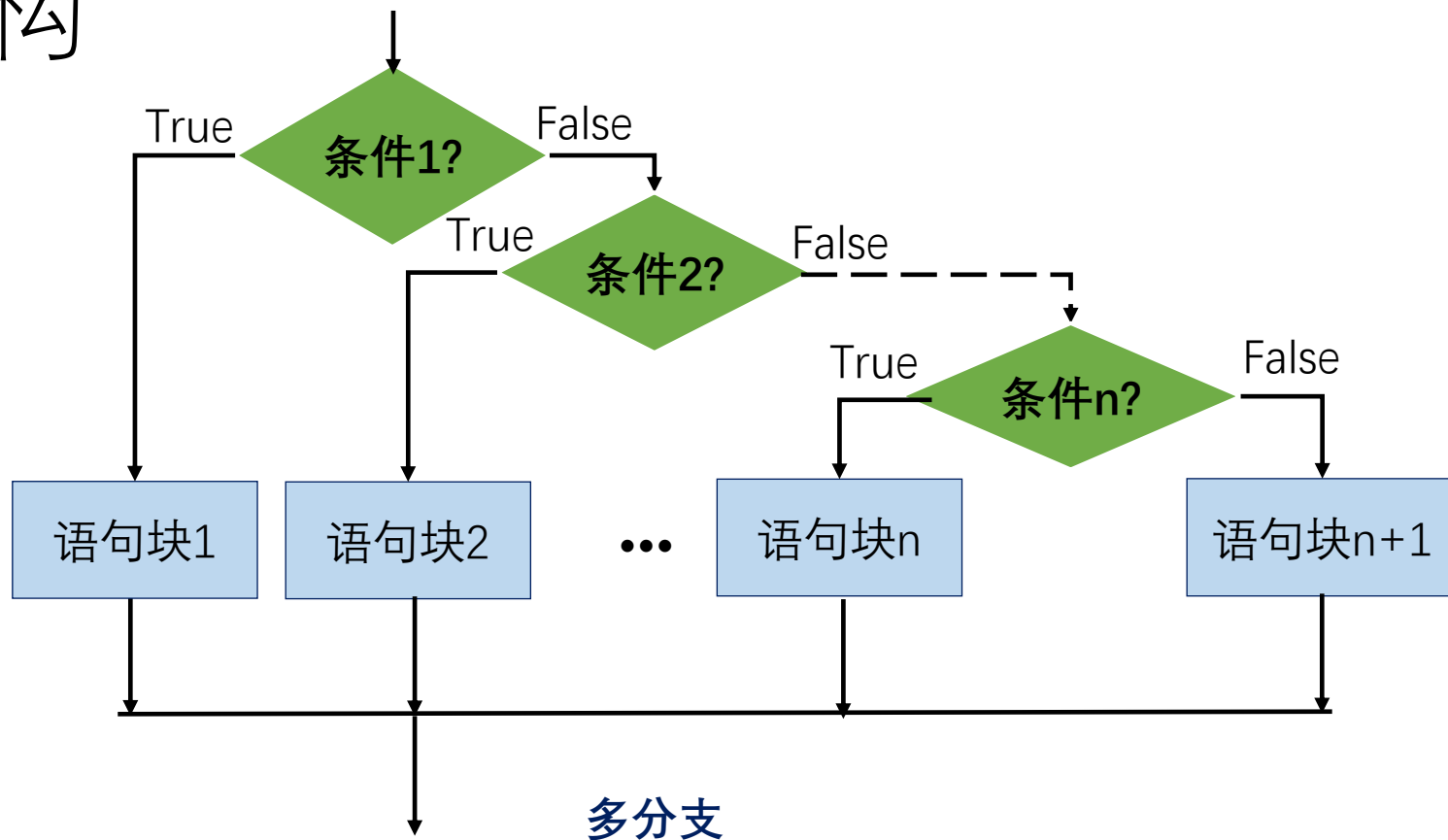
```
def grade(score):  
    if score > 100 or score < 0:  
        return 'wrong score.must in [0,100].'  
    else:  
        if score >= 90:  
            return 'A'  
        else:  
            if score >= 80:  
                return 'B'  
            else:  
                if score >= 70:  
                    return 'C'  
                else:  
                    if score >= 60:  
                        return 'D'  
                    else:  
                        return 'F'
```



```
def grade2(score):  
    degree = 'DCBAAF'  
    if score > 100 or score < 0:  
        return 'wrong score.must  
between 0 and 100.'  
    else:  
        index = (score - 60)//10  
        if index >= 0:  
            return degree[index]  
        else:  
            return degree[-1]
```

3.2.3 多分支结构

```
if 条件1:  
    语句块1  
elif 条件2:  
    语句块2  
elif 条件3:  
    语句块3  
else:  
    语句块4
```



其中，关键字elif是else if的缩写。

条件i 实际上等于 前面的条件都为假 再加上条件i

复旦大学版权所有

3.2.3 多分支结构

```
def grade3(score):  
    if score > 100 or score < 0:  
        return 'wrong score.must in [0,100].'  
    elif score >= 90:  
        return 'A'  
    elif score >= 80:  
        return 'B'  
    elif score >= 70:  
        return 'C'  
    elif score >= 60:  
        return 'D'  
    else:  
        return 'F'
```

```
def grade1(score):  
    if score > 100 or score < 0:  
        return 'wrong score.must in [0,100].'  
    else:  
        if score >= 90:  
            return 'A'  
        else:  
            if score >= 80:  
                return 'B'  
            else:  
                if score >= 70:  
                    return 'C'  
                else:  
                    if score >= 60:  
                        return 'D'  
                    else:  
                        return 'F'
```



score >=70 实际上为 70<=score<80

复旦大学版权所有

3.2.5 选择结构应用

- 例1：面试资格确认。

如果为电子信息工程专业(major) 学生, 年龄(age)在25岁以上或者重点院校(college)
如果为计算机专业, 年龄在28岁以下

```
age=24
subject="计算机"
college="非重点"
if (age > 25 and subject=="电子信息工程") or (college=="重点" and subject=="电子信息工程") \
    or (age<=28 and subject=="计算机"):
    print("恭喜, 你已获得我公司的面试机会!")
else:
    print("抱歉, 你未达到面试要求")
```

练习！

```
(subject == '电子信息工程' and (age> 25 or college == '重点')) or (age<=28 and subject=="计算机")
```

3.2.5 选择结构应用*(while循环语句再讲)

- 例2：用户输入若干个分数，求所有分数的和。每输入一个分数后询问是否继续输入下一个分数，回答“yes”就继续输入下一个分数，回答“no”就停止输入分数。

```
endFlag = 'yes'
s = 0
while endFlag.lower() == 'yes':
    x = input("请输入一个正整数: ")
    x = eval(x)
    if isinstance(x, int) and 0<=x<=100:
        s = s + x
    else:
        print('不是数字或不符合要求')
    endFlag = input('继续输入? (yes or no) ')
print('整数之和=', s)
```

练习！

3.2.5 选择结构应用

例3：编写程序，判断今天是今年的第几天。

1. 首先得到今天对应的年份、月份和当月第几天
2. 今年的第几天=前面的月份天数之和+当月第几天

2月的天数随闰年而不同, 怎么判断闰年？

- 能被4整除, 但不能被100整除
- 能被400整除

```
import time
date = time.localtime()
year = date[0]
month = date[1]
day = date[2]
day_month = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
if year % 400 == 0 or (year % 4 == 0 and year % 100 != 0): #判断是否为闰年
    day_month[1] = 29
if month==1:
    print(day)
else:
    print(sum(day_month[:month-1])+day)
```

>>>print(time.localtime()[-2])

练习！

模块time

- 对于时间进行不同格式之间的转换
- 两种格式：
 - 从epoch(即标准时1970年1月1日) 开始到现在的秒数 (浮点数)
 - 包含9个整数的tuple (年,月,日,时钟,分钟,秒,星期几,当年第几天, 是否夏时制)
- 时间有关的模块还包括datetime和calendar等

```
>>> import time
>>> time.time()
1492011487.5653772
>>> time.localtime()
time.struct_time(tm_year=2017, tm_mon=4, tm_mday=12, tm_hour=15, tm_min=36, tm_sec=29,
tm_wday=2, tm_yday=102, tm_isdst=0)
>>> time.asctime(time.localtime())
'Wed Apr 12 23:38:31 2017'
```

常见错误：错误递进

```
import math
radius = -20
if radius >= 0:
    area = radius * radius * math.pi
print("The area is", area)
```



```
import math
radius = -20
if radius >= 0:
    area = radius * radius * math.pi
print("The area is", area)
```

```
i = 1
j = 2
k = 3
if i > j:
    if i > k:
        print('A')
    else:
        print('B')
```



```
i = 1
j = 2
k = 3
if i > j:
    if i > k:
        print('A')
else:
    print('B')
```

常见错误：条件表达式错误

```
def grade3(score):  
    if score > 100 or score < 0:  
        return 'wrong score.must in [0,100].'  
    elif score >= 90:  
        return 'A'  
    elif score >= 80:  
        return 'B'  
    elif score >= 70:  
        return 'C'  
    elif score >= 60:  
        return 'D'  
    else:  
        return 'F'
```

```
def grade4(score):  
    if score > 100 or score < 0:  
        return 'wrong score.must in [0,100].'  
    elif score >= 0:  
        return 'F'  
    elif score >= 60:  
        return 'D'  
    elif score >= 70:  
        return 'C'  
    elif score >= 80:  
        return 'B'  
    else:  
        return 'A'
```

注意条件的先后顺序

复旦大学版权所有

grade4(94)也会返回'F'

谢谢

复旦大学版权所有